

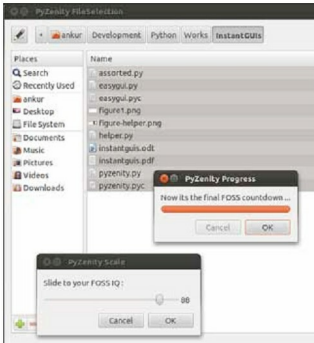


Figure 3: Assorted dialog boxes provided by PyZenity

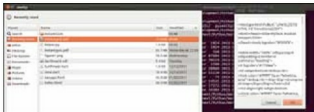


Figure 4: PyZenity helper function dialog boxes

```

return
else:
    msg3 = 'How long do you wanna live with FOSS?'
    yrs = integerbox(msg3, 'EasyGUI Integerbox',
                    lowerbound = 1, upperbound = 500)

    if None != yrs:
        msg = 'You run %s and will live with FOSS for %s
years, long live FOSS and Geekz !!!' % (vals[0], str(yrs))
        textbox('Greetings from FOSS community', 'EasyGUI
Greetings', msg)

if '__main__' == __name__:
    main()

```

The EasyGUI class *EgStore* lets you save and retrieve persistent data. To explore the documentation further, click *Help* in the EasyGUI demo. In fact, the demo is the best example of its usage; read its source for working code examples.

Python meets Zenity with PyZenity

Zenity is a seasoned and well-known command-line utility that creates various GTK+ dialogue boxes to add GUIs

to shell scripts. It comes pre-installed with Ubuntu, and you can create very useful applications with its dialogue boxes knitted together. One example is the Ubuntu Customisation Kit (a tool to customise various flavours of Ubuntu live CDs). PyZenity is an object-oriented wrapper over Zenity to expose its simplicity and power to Python. PyZenity is a single Python file; you need the subversion tool to get a copy from its Google Code repository. To install Subversion, run `sudo apt-get install subversion` and then follow the link in *References* to get a read-only copy of *PyZenity*. Just copy *pyzenity.py* from the *pyzenity* checkout directory to your application source folder. You can also install *pyzenity* with `sudo python setup.py install` in the *pyzenity* checkout directory, for system-wide availability. Please note that the Google Code version is not very recent, but it's easy to add features from the latest releases, as *pyzenity.py* is only around 454 lines. Recent versions of Zenity have additional colour selection, forms and password dialogue boxes that are missing from *pyzenity.py* but these could be added by following the existing dialogue box class code. PyZenity provides the classes in Table 2, to wrap almost all Zenity functionality.

Table 2: PyZenity classes

<i>ZCalendar</i>	Creates a calendar dialogue box
<i>ZEntry</i>	An entry dialogue—visible, or password-style hidden text
<i>ZError</i>	An error notification dialogue box
<i>ZInfo</i>	An information notification dialogue box
<i>ZFileSelection</i>	Single or multiple file selection dialogue box
<i>ZList</i>	Check-list or radio-list style dialogue box
<i>ZNotification</i>	Task-bar notification text
<i>ZProgress</i>	Dialogue with progress bar
<i>ZQuestion</i>	A dialogue box requiring an answer to a query
<i>ZText</i>	Lets one read and modify text of a file
<i>ZWarning</i>	Warning dialogue box
<i>ZScale</i>	A dialogue box to accept a value via a slider widget

To see it in action, download *assorted.py* from http://www.linuxforu.com/article_source_code/may12/instantgui_source_code.zip and run it (`python assorted.py`) in a text console. Figure 3 shows some Zenity dialogue boxes created by *assorted.py*.

Here is some theory to help you use PyZenity more efficiently. In its object-oriented design, all the classes shown in Table 2 are derived from *ZenityBase*, because